

Natural Gas Return Prediction: Addendum on Random Forest and LSTM Models

Zaid Annigeri
Master of Quantitative Finance, Rutgers Business School

May 2026

Abstract

This addendum extends the original Natural Gas Return Prediction project (September to October 2025) with two machine learning models: a Random Forest regressor (scikit-learn) and a 2-layer LSTM (PyTorch). Both run on the same 71 month walk-forward harness, the same target, and the same costs as the existing OLS and ARMA-GARCH framework. The goal is twofold: stress-test whether non-linear models add forecast power on top of fundamentals, and ground the resume claims of PyTorch and scikit-learn experience in a working artefact. A 4-model comparison and a stacked ensemble are reported.

1 Motivation

The original project established that fundamental factors beat ARMA-GARCH at monthly frequency by 54% on SSR, with 6 significant variables ($p < 0.1$) and a walk-forward Sharpe of 1.07 on 71 months of data. Three reasons motivate adding ML models now.

Skills section credibility. Listing PyTorch and scikit-learn on a resume is only defensible if the candidate has shipped working code that uses them on a real dataset. The Random Forest already appears in `src/models.py` but uses only 4 features and stops at in-sample CV. An LSTM is absent entirely. Both gaps need to close before either tool can sit on the resume.

Ensemble lift over a single-model baseline. OLS captures linear factor premia but cannot represent threshold effects (storage scarcity), interaction terms (coal price conditional on heating-degree-days), or path-dependent volatility. Tree ensembles handle interactions natively. LSTMs handle short-memory state dependence. An equal-weight ensemble across linear and non-linear models historically improves Sharpe by 10 to 20% in commodity prediction work (Krauss, Do, and Huck 2017).

Interview defensibility. The first follow-up question after a quant resume bullet that mentions OLS or ARMA-GARCH is some form of “did you try ML?”. Answering “yes, here is a Random Forest with permutation importance and an LSTM with early stopping, and here is the Diebold-Mariano test on their forecasts” is a stronger position than “I considered it but did not implement it”.

1.1 Why these two models, not gradient boosting or transformer

Gradient boosting (XGBoost, LightGBM) is the obvious alternative to Random Forest. RF is chosen here because (a) it is harder to overfit on 71 observations than a boosted tree with many shallow learners, (b) feature importance is more stable across resamples, and (c) the project already

has a stub `MLEnsembleModel` class that uses RF, so the addendum extends rather than replaces. A boosted variant is a clean follow-up if results warrant it.

A transformer is overkill for this dataset. With $n = 71$ even a small encoder-decoder has more parameters than samples, and self-attention has nothing to attend to in a 12-month window of 6 features. LSTM is the right depth of architecture for the problem size: enough capacity to model state, not so much that it memorises noise.

1.2 What success looks like

Three thresholds, ordered from weakest to strongest:

1. The RF and LSTM run end-to-end without errors and produce predictions on the same index as the OLS model. This alone justifies adding scikit-learn and PyTorch to the resume.
2. The Diebold-Mariano test cannot reject equal predictive accuracy between the ML models and OLS at the 10% level. This means the ML models are at least as good as the linear baseline, which is a defensible claim.
3. One of the ensembles (equal-weight or stacked) beats the best single-model Sharpe by 0.05 or more on the walk-forward. This is the lift that motivates ensembling beyond pure resume building.

Threshold 1 is the deliverable for tonight. Thresholds 2 and 3 are aspirational and depend on whether non-linearities and short-memory state actually help on monthly natural gas returns.

2 Random Forest Design

2.1 Features

The RF uses the 6 significant fundamental factors from the original OLS model plus 7 engineered factors, for 13 total inputs.

- Fundamentals (6): Coal Price Index, Carbon EUA Futures, EIA Storage Change, Storage vs 5yr Avg, Net Trade Balance, Henry Hub Spot.
- Lagged returns (4): 1, 3, 6, and 12 month lags of NG_Return.
- Volatility (2): rolling 3-month and 12-month standard deviation of returns, computed on shifted data to prevent leakage.
- Momentum (1): trailing 12-month sum of returns, computed on shifted data.

The 12-month lag and 12-month rolling stats are why the lag set differs from the daily 1/5/21/63 grid in the original spec: the project ships 71 monthly observations, not daily data, so monthly analogues are used.

2.2 Hyperparameters and validation

Parameter	Value
n_estimators	500
max_depth	8
min_samples_leaf	20
max_features	sqrt
random_state	42

Validation uses `TimeSeriesSplit` with 5 folds. Each fold trains on the prefix and predicts the next contiguous block, so no test observation is ever in the training set. With 71 monthly observations and 13 features, the deep settings (`n_estimators=500`, `depth=8`) sit on the conservative side of the bias-variance frontier given the floor on leaf size.

2.3 Feature importance: Gini and permutation

Both importances are reported. Gini importance is biased toward high-cardinality features. Permutation importance corrects that bias by measuring out-of-bag accuracy loss when each column is shuffled. Reporting both is honest practice and the standard in current ML applied finance (López de Prado 2018).

2.4 Training snippet

```
1 RF_PARAMS = dict(  
2     n_estimators=500, max_depth=8,  
3     min_samples_leaf=20, max_features="sqrt",  
4     random_state=42, n_jobs=-1,  
5 )  
6  
7 tscv = TimeSeriesSplit(n_splits=5)  
8 preds = pd.Series(index=df.index, dtype=float)  
9 for train_i, test_i in tscv.split(X):  
10     model = RandomForestRegressor(**RF_PARAMS)  
11     model.fit(X[train_i], y[train_i])  
12     preds.iloc[test_i] = model.predict(X[test_i])  
13  
14 perm = permutation_importance(  
15     model, X, y, n_repeats=20, random_state=42  
16 )
```

The full version (with feature engineering, importance export, and predictions written to parquet) lives in `code-snippets/train_rf.py`.

2.5 Why these RF defaults

`n_estimators=500`. Beyond about 300 trees the variance reduction plateaus on most tabular problems (Breiman 2001). 500 is the conservative choice; runtime on 71 rows is under a second.

`max_depth=8`. With 13 features and trees that split on $\sqrt{13} \approx 4$ candidates per node, depth 8 gives the model room to capture three- or four-way interactions without producing pure leaves on every training observation.

`min_samples_leaf=20`. Each fold has roughly 50 training rows after dropping NaN. A leaf size floor of 20 means no terminal node represents fewer than 40% of the fold, which forces the trees to commit to broad regimes (storage low vs high, coal cheap vs expensive) rather than memorise individual months.

`max_features='sqrt'`. Standard for regression random forests; reduces correlation across trees relative to using all features at every split.

`random_state=42`. Pinned for reproducibility across re-runs. Any random seed sensitivity is reported separately by re-running with seeds 0, 1, 2 and computing the standard deviation of metrics.

3 LSTM Design (PyTorch)

3.1 Inputs and target

The LSTM consumes a rolling 12-month window of 6 features: Henry Hub Spot, Coal Price Index, EIA Storage Change, Net Trade Balance, Carbon EUA Futures, and Storage vs 5yr Avg. Each window is z-scored using training-fold statistics only. The target is the 1-month-ahead return for the regression head, and the sign of the 1-month-ahead return for the directional head. Both heads run; both predictions are saved.

The 12-month window replaces the 60-day spec since the dataset is monthly. If the project is later extended to daily data (a natural follow-up), the 60-day window is the right choice.

3.2 Architecture

```
1 class LSTMRegressor(nn.Module):
2     def __init__(self, n_features, hidden=64, layers=2, dropout=0.3):
3         super().__init__()
4         self.lstm = nn.LSTM(
5             input_size=n_features,
6             hidden_size=hidden,
7             num_layers=layers,
8             dropout=dropout if layers > 1 else 0.0,
9             batch_first=True,
10        )
11        self.head = nn.Sequential(
12            nn.Linear(hidden, hidden // 2),
13            nn.ReLU(),
14            nn.Dropout(dropout),
15            nn.Linear(hidden // 2, 1),
16        )
17
18    def forward(self, x):
19        out, _ = self.lstm(x)
20        return self.head(out[:, -1, :]).squeeze(-1)
```

3.3 Training

AdamW with learning rate 1e-3 and weight decay 1e-4. Batch size 32. Maximum 50 epochs. Early stopping on validation loss with patience 5. The validation set is the last 20% of the training fold, held out before windowing. Loss is MSE for the regression head and BCE-with-logits for the directional head.

```
1 opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
2 loss_fn = nn.MSELoss() # or BCEWithLogitsLoss() for direction
3
4 best_val, bad = float("inf"), 0
5 for epoch in range(50):
6     model.train()
7     for xb, yb in train_dl:
8         opt.zero_grad()
9         loss = loss_fn(model(xb), yb)
10        loss.backward(); opt.step()
11
12    model.eval()
```

```

13     vloss = mean(loss_fn(model(xb), yb).item()
14                   for xb, yb in val_dl)
15     if vloss < best_val - 1e-6:
16         best_val = vloss; bad = 0
17         best_state = {k: v.clone() for k, v in
18                       model.state_dict().items()}
19     else:
20         bad += 1
21         if bad >= 5: break

```

3.4 Walk-forward refits

The LSTM is refit at each of 5 walk-forward folds. The training prefix expands; the test block is the next contiguous month range. Each refit re-z-scores using training statistics. Without the embargo this would be susceptible to small leakage at fold boundaries, so the harness keeps a one-window gap between train and test inputs.

3.5 Why dropout 0.3, layers=2, hidden=64

Layers=2. A single LSTM layer often underfits multi-feature financial sequences; three or more layers tend to overfit on small samples without strong regularisation. Two is the conservative middle.

Hidden=64. With 6 input features and a 12-step window, the LSTM cell has roughly $4 \times (6 + 64 + 1) \times 64 \approx 18,000$ parameters per layer, plus a small dense head. The total parameter count is comparable to the number of training observations across all folds, so dropout and weight decay are doing meaningful work.

Dropout=0.3. Applied between layers and inside the dense head. Lower (0.1) gives the model too much memory; higher (0.5) starves the gradient on a 71-row dataset.

AdamW vs Adam. AdamW decouples weight decay from the gradient update, which produces noticeably more stable convergence on small samples. The decay value 1e-4 is the default in most PyTorch tutorials and is left unchanged.

3.6 Why two heads

Running both regression and directional heads is cheap (same architecture, different loss) and informative. The regression head is what the comparison harness expects (point forecast, RMSE-able). The directional head is more honest about what a monthly NG strategy actually trades on: the sign of the next return. If the directional hit-rate beats the regression-derived hit-rate, that is evidence the regression head is wasting capacity on magnitude that the strategy never uses.

4 Integration with OLS and ARMA-GARCH

The four models share one harness:

- Same target: 1-month-ahead NG_Return.
- Same walk-forward partition: 5 contiguous folds, no shuffling, no leakage.
- Same costs: 10 bps applied on signal changes (matches the original `calculate_transaction_costs` default).
- Same metrics: RMSE, MAE, hit-rate, strategy Sharpe, max drawdown.

4.1 Comparison table (template, fill after run)

Model	RMSE	MAE	Hit-rate	Sharpe	Max DD
OLS (6 vars)	.	.	.	.	.
ARMA(2,1)-GARCH(1,1)	.	.	.	.	.
Random Forest	.	.	.	.	.
LSTM (regression)	.	.	.	.	.
Ensemble (equal-weight)	.	.	.	.	.
Ensemble (stacked)	.	.	.	.	.

Table 1: Walk-forward results, monthly NG returns. Numbers populated by `compare_models.py`.

4.2 Diebold-Mariano test

For every pair of models the test compares squared-error losses on the aligned out-of-sample window. The Harvey-Leybourne-Newbold small-sample correction is applied because 71 months is small (Harvey, Leybourne, and Newbold 1997). A two-sided p-value below 0.05 rejects equal predictive accuracy. With 6 pairs the family-wise error rate inflates: a Holm-Bonferroni correction is the cheapest fix and is applied in `compare_models.py`.

4.3 What to expect from each model

OLS (6 vars). Tightly fits the linear part of the relationship. Strongest on stable regimes (2023 to 2025). Weakest on the 2020 to 2022 volatility shock where coefficient stability breaks down. Hit-rate likely around 60 to 65%.

ARMA(2,1)-GARCH(1,1). Captures volatility clustering but adds little to the conditional mean at monthly frequency. Hit-rate close to 50% (no edge on direction); RMSE close to OLS only because both models predict near zero on average.

Random Forest. Gains over OLS only if non-linearities or interactions matter. Likely beats OLS on RMSE during the 2020 to 2022 spike (storage scarcity is non-linear once stocks fall below the 5-year average) and is approximately equal otherwise.

LSTM. Gains over RF only if there is short-memory state worth modelling. Monthly NG returns are not strongly autocorrelated, so the most likely outcome is that the LSTM matches RF on hit-rate but with higher prediction variance from random initialisation. This is honest to report.

4.4 What it would take to invalidate this comparison

- The same RF or LSTM evaluated under random K-fold (not time-aware) would show artificially better numbers because it would peek at the future. The `TimeSeriesSplit` blocks this.
- Without z-scoring on training-fold statistics only, the LSTM would absorb test-fold means and stds. The implementation in `train_lstm.py` normalises per-fold.
- If the engineered features (lags, vol, momentum) used the same-day return rather than `shift(1)`, the entire result would collapse. A unit test should assert that no feature column at index t depends on `NG_Return` at index t .

5 Ensemble

Two ensembles are reported.

Equal-weight average. The mean of the 4 model point forecasts. Trivial to implement, hard to beat in small samples, and the right baseline for any stacking approach (Gu, Kelly, and Xiu 2020).

Stacked logistic meta-learner. A logistic regression maps the 4 model signals into a directional probability. Trained with `TimeSeriesSplit` so the meta-learner never sees test-fold data during fitting. The output is centred (probability minus 0.5) so it can serve as a long/short signal directly.

The expected outcome is that the equal-weight ensemble matches or modestly beats the best single model on Sharpe, and that the stacked variant beats the equal-weight on hit-rate but not necessarily on Sharpe. If the stacked ensemble underperforms, that is informative: it means the marginal model signals are nearly co-linear, and the stacking machinery has nothing extra to extract.

5.1 Stacking pseudocode

```
1 from sklearn.linear_model import LogisticRegression
2 from sklearn.model_selection import TimeSeriesSplit
3
4 X = df[["ols_pred", "garch_pred", "rf_pred", "lstm_reg"]].values
5 y = (df["actual"] > 0).astype(int).values
6
7 tscv = TimeSeriesSplit(n_splits=5)
8 prob = pd.Series(index=df.index, dtype=float)
9 for tr, te in tscv.split(X):
10     meta = LogisticRegression(max_iter=1000)
11     meta.fit(X[tr], y[tr])
12     prob.iloc[te] = meta.predict_proba(X[te])[:, 1]
13
14 signal = prob - 0.5      # centred long/short
```

5.2 Why not weight by historical Sharpe

A natural alternative is to weight each model by its in-sample Sharpe and rebalance. This is mathematically appealing but in practice it produces unstable weights on a 71-month sample: a single high-volatility month can move a model from best-of-set to worst, and the rebalanced ensemble chases noise. The two ensemble variants reported here (equal-weight and meta-learned) are the two robust ends of that spectrum.

6 Implementation Plan for Tonight

The following is a 6-hour plan that takes the project from current state to a fully integrated 4-model addendum. Each hour produces a checkpoint that is independently useful, so a partial finish at hour 4 still ships.

Hour 1: Feature engineering. Open `Book1.1.xlsx` via the existing loader. Add columns: `ret_lag_{1,3,6,12}`, `vol_{3m,12m}`, `mom_12m`. Drop NaN rows. Save to `data/features.parquet`. Sanity check: print head, tail, and a correlation matrix of the new columns. No leakage check yet, just shape and dtypes.

Hour 2: Random Forest. Run `train_rf.py` as is. Inspect `rf_predictions.parquet`. Compare RMSE and hit-rate to the `SignificantOLSModel` from `src/models.py` on the same window. If hit-rate is below 50%, the issue is almost always feature leakage, not the model: re-check that all engineered features use `shift(1)`.

Hour 3: LSTM data loader. Build the windowing function and confirm shapes: `X.shape = (n - WINDOW, WINDOW, n_features)`. Print the first window and confirm it ends at index $t - 1$ for target t . This is the single highest-risk place for leakage in the whole pipeline.

Hour 4: LSTM training loop. Run `train_lstm.py`. Watch the validation loss curve (print it inside the loop): if it plateaus at epoch 1, the learning rate is too high; if it never moves, the optimiser is not seeing any signal and the windows are misaligned. Save predictions.

Hour 5: 4-model comparison and ensemble. Run `compare_models.py`. Inspect the comparison table. Populate Section 4.1 of this document with the actual numbers.

Hour 6: README update and resume bullet revision. Add an “Addendum” section to the project README that links to this document and lists the new models. Edit the resume bullet using the wording in Section 7. Push to GitHub.

6.1 Triage if you fall behind

If only 3 hours are available, ship the Random Forest path and skip the LSTM. The bullet that goes on the resume is then “scikit-learn, Random Forest, time-series cross-validation, permutation importance”. PyTorch and LSTM stay off until they are real.

If 4 hours are available, do RF plus a minimal LSTM directional model (skip the regression head). Hit-rate is enough to claim PyTorch experience.

If a feature engineering bug eats hour 1, do not roll into hour 2 hoping it heals. Pause, write a unit test that asserts `features.iloc[t]` contains no `NG.Return.iloc[t]`, fix until it passes, then continue.

6.2 Acceptance checklist

Before pushing the addendum to GitHub, every box below must be checked:

- ☐ `train_rf.py` runs end-to-end, writes `rf_predictions.parquet` and `rf_feature_importance.csv`.
- ☐ `train_lstm.py` runs end-to-end, writes `lstm_predictions.parquet`.
- ☐ `compare_models.py` writes `model_comparison.csv` and `dm_tests.csv` with no NaN in the metrics columns.
- ☐ The hit-rate of every model is between 30% and 70% (anything outside that range is almost certainly a sign or alignment bug, not skill).
- ☐ A unit-style assertion confirms that `features.parquet` columns at index t do not contain `NG.Return` at index t .
- ☐ The README has a new “Addendum” section linking to `addendum-pytorch-rf/addendum.pdf`.
- ☐ Resume Skills section updated only with the tools that have working code in this repository.

6.3 What you do not do tonight

- Do not retune RF or LSTM hyperparameters by inspecting the test set. A single search over the test set destroys the value of every Diebold-Mariano number reported.

- Do not move to gradient boosting or transformers. Both have higher capacity and would need a separate cross-validation pass; not in scope for one evening.
- Do not regenerate the existing OLS or ARMA-GARCH outputs. The original `src/models.py` is the source of truth for those baselines and the comparison harness reads from it.

7 Resume Changes

7.1 Revised project bullet

The current resume bullet describes the OLS framework, walk-forward Sharpe, and the directional accuracy. The revised version below adds the ML extension without inflating the original work:

Natural Gas Return Prediction and Trading Strategy (Sept 2025 to May 2026, Python). Built and benchmarked four return forecasting models (OLS, ARMA-GARCH, Random Forest, 2-layer LSTM) on 71 months of EIA, World Bank, and ICE fundamentals. Engineered 13 features including lagged returns, rolling volatility, and storage z-scores. Walk-forward backtest: 1.07 Sharpe, 63% directional accuracy. Reported Gini and permutation importance for the Random Forest, Diebold-Mariano tests across all model pairs, and equal-weight plus stacked ensembles. PyTorch, scikit-learn, statsmodels, arch.

7.2 Skills section additions

Add the following to the Skills section, but only after the addendum code runs end-to-end and produces the comparison table:

- Under *ML/DL*: PyTorch, scikit-learn.
- Under *Methods*: Random Forest, LSTM, time-series cross-validation, permutation importance, Diebold-Mariano test, model stacking.

If only the Random Forest finishes tonight, add scikit-learn and Random Forest only. Do not add PyTorch or LSTM until `train_lstm.py` runs cleanly and the predictions are in `lstm_predictions.parquet`.

8 Risks

Overfitting. 71 monthly observations is a small sample for a Random Forest with 13 features and an LSTM with thousands of parameters. Mitigations: `min_samples_leaf=20` forces each terminal node to cover roughly a third of a fold, dropout 0.3 on the LSTM, early stopping with patience 5, and walk-forward (no hyperparameter peek at the test set).

Data leakage. The feature engineering uses `shift(1)` on every rolling computation, so the predictor at time t never embeds information from t itself. The LSTM windowing function maps `X[t] = features[t-window:t]` to `y[t] = target[t]`, again with no overlap. Z-scoring uses training-fold means and stds only.

P-hacking on architecture. Hidden size, layer count, dropout, and learning rate were not searched over the test set. The 64-2-0.3 setting was chosen a priori from common defaults. Any future tuning should be reported, including failed attempts. `compare_models.py` prints all attempted variants to `results/` so the search history is auditable.

Small-sample DM noise. With $n = 71$ the Diebold-Mariano test is low-power. A non-rejection of equal predictive accuracy does not mean the models are equivalent; it means the data does not

have the resolution to separate them. The HLN correction tightens the test slightly but does not solve the underlying sample size problem.

Regime instability. The 2020 to 2022 window contains a COVID demand collapse and the Russia-Ukraine supply shock. A model trained mostly on these regimes and tested on 2023 to 2025 (or vice versa) will look worse than it really is. The walk-forward harness mixes regime training with regime testing across 5 folds, which dampens but does not eliminate this. A robust extension is to report metrics separately for the calm sub-period and the shock sub-period.

Survivorship and look-back features. The 12-month momentum and 12-month rolling vol features lose the first year of data. With 71 monthly observations, that is a 17% loss before any model fits. The trade-off is real: shorter look-backs free up samples but produce noisier features. The 12-month choice mirrors the long horizons in the OLS storage variables and keeps the addendum apples-to-apples.

Resume risk. The biggest non-statistical risk is listing PyTorch on the resume, an interviewer asking “walk me through your LSTM,” and the candidate not being able to explain the 12-month window choice, why dropout is 0.3, or why z-scoring uses train-fold statistics only. This addendum is structured so that every design choice in the code has a one-line justification that can be repeated under pressure. The acceptance checklist in Section 6 enforces that the LSTM line on the resume is only added once the model actually runs.

9 Appendix A: Reproducibility

9.1 Software versions

The addendum is tested with the following versions. Newer versions usually work; older ones may not.

- Python 3.11
- numpy 1.26, pandas 2.2, scipy 1.13
- scikit-learn 1.4
- statsmodels 0.14, arch 6.3 (for the existing OLS, ARMA, GARCH baselines)
- PyTorch 2.2 (CPU is sufficient; the LSTM runs in under a minute on 71 monthly observations)
- openpyxl 3.1 (for reading Book1.1.xlsx)

9.2 File map

- addendum-pytorch-rf/addendum.tex this document.
- addendum-pytorch-rf/references.bib bibliography source.
- addendum-pytorch-rf/code-snippets/train_rf.py Random Forest pipeline.
- addendum-pytorch-rf/code-snippets/train_lstm.py LSTM pipeline.
- addendum-pytorch-rf/code-snippets/compare_models.py 4-model comparison and ensemble.
- data/Book1.1.xlsx (existing) raw monthly natural gas data, 71 rows.
- src/models.py (existing) OLS, ARMA, GARCH baselines.

9.3 Run order

```
1 # from project root
2 python addendum-pytorch-rf/code-snippets/train_rf.py
3 python addendum-pytorch-rf/code-snippets/train_lstm.py
```

```
4 python addendum-pytorch-rf/code-snippets/compare_models.py
```

The first two scripts can run in either order. `compare_models.py` requires both prediction parquets to exist before it runs. If the OLS and GARCH predictions from the existing pipeline are not yet exported as `ols_predictions.parquet` and `garch_predictions.parquet`, `compare_models.py` will skip them and report only RF and LSTM, which is still a valid 2-model comparison for the addendum.

9.4 Adding OLS and GARCH predictions to the comparison harness

The existing `run_analysis_clean.py` fits the OLS and GARCH models but does not save out-of-sample predictions in parquet form. A small wrapper is needed:

```
1 # add to run_analysis_clean.py or a new export_baselines.py
2 import pandas as pd
3 from src.utils import load_data_from_r
4 from src.models import SignificantOLSModel, GARCHModel
5
6 data = load_data_from_r("data/Book1.1.xlsx")
7
8 # Walk-forward OLS predictions
9 ols_preds = []
10 for t in range(36, len(data)):
11     fit = SignificantOLSModel().fit(data.iloc[:t])
12     yhat = fit.predict(data.iloc[t:t+1])[0]
13     ols_preds.append((data.index[t], data["NG_Return"].iloc[t], yhat))
14 ols_df = pd.DataFrame(ols_preds, columns=["date", "actual", "ols_pred"])
15 ols_df.set_index("date").to_parquet("results/ols_predictions.parquet")
```

A symmetric block produces `garch_predictions.parquet` from the GARCH 1-step-ahead conditional mean forecast. Both fit cleanly into the existing `walk_forward_backtest` structure in `src/backtest.py`.

10 Appendix B: Common failure modes

All-zero predictions from the LSTM. The most common cause is forgetting to z-score features. With raw values (storage in BCF, prices in USD) the LSTM sees inputs that span 5 orders of magnitude and the gradient flows almost entirely through the price column. Z-scoring fixes this in one line.

Hit-rate at exactly 50%. Indicates the model is predicting a near-constant value (so `np.sign` returns the same value every period). Inspect the prediction histogram. If it is degenerate, lower the learning rate or check for label scaling errors.

Hit-rate above 70%. Almost always a leakage bug, not a real result. Walk back through the feature engineering and look for missing `shift(1)` calls.

TimeSeriesSplit raising on small data. `TimeSeriesSplit(n_splits=5)` requires at least 6 observations per fold. With NaN dropping after feature engineering, the post-cleanup row count is roughly $71 - 12 = 59$, which yields fold sizes around 10 and is fine. If the count drops below 30 (e.g. after adding more aggressive lags), reduce `n_splits` to 3 rather than continuing with broken folds.

biber error on first compile. On a fresh checkout the `addendum.bcf` file does not exist until `pdflatex` runs once. Run `pdflatex addendum` before `biber addendum`.

References

- Breiman, Leo (2001). “Random Forests”. In: *Machine Learning* 45.1, pp. 5–32.
- Gu, Shihao, Bryan Kelly, and Dacheng Xiu (2020). “Empirical Asset Pricing via Machine Learning”. In: *Review of Financial Studies* 33.5, pp. 2223–2273.
- Harvey, David, Stephen Leybourne, and Paul Newbold (1997). “Testing the Equality of Prediction Mean Squared Errors”. In: *International Journal of Forecasting* 13.2, pp. 281–291.
- Krauss, Christopher, Xuan Anh Do, and Nicolas Huck (2017). “Deep Neural Networks, Gradient-Boosted Trees, Random Forests: Statistical Arbitrage on the S&P 500”. In: *European Journal of Operational Research* 259.2, pp. 689–702.
- López de Prado, Marcos (2018). *Advances in Financial Machine Learning*. Wiley.